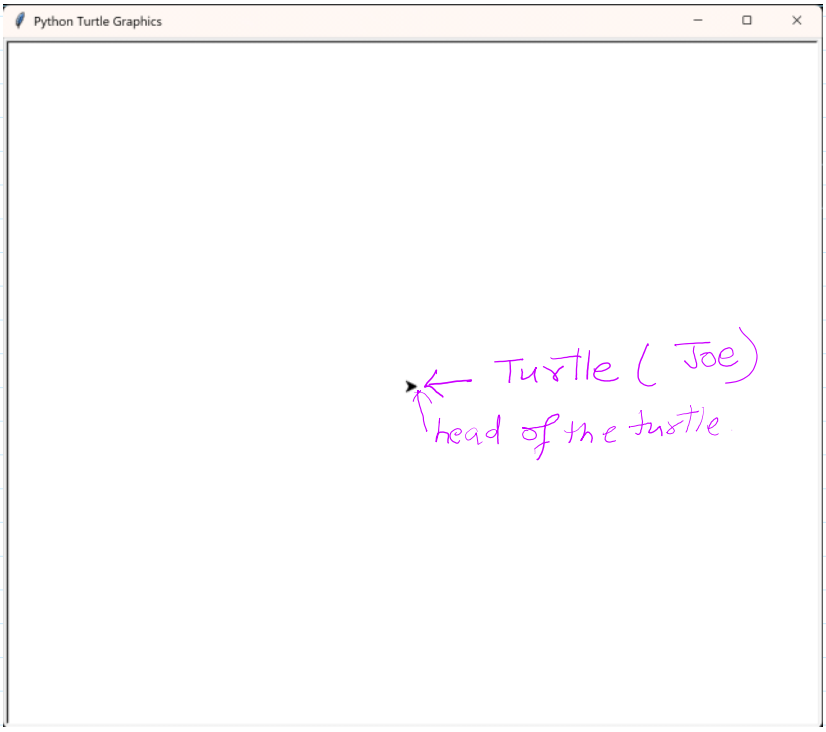
Turtle

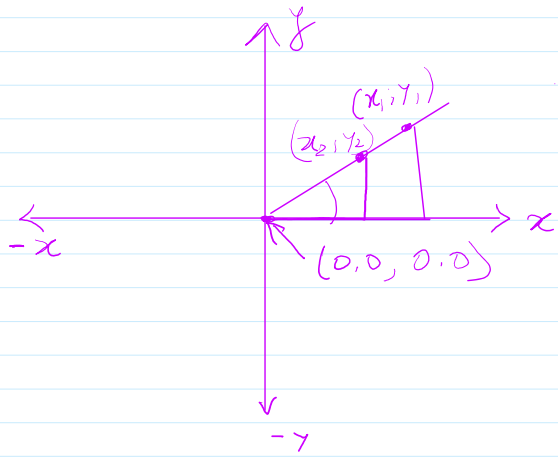
05 August 2025 20:05

What is Turtle in Python?
Turtle is a built in Python module that enables simple graphics and drawings. It is widely used to teach programming concepts visually by moving cursor (turtle) that can draw on the screen.

```python
import turtle as T
Joe = T.Turtle()
T.done()
```

Output:





BASIC COMMANDS

- forward(x) or fd(x): Moves forward x units

```
def fd(
    self: Self@TNavigator,
    distance: float
) -> None
Move the turtle forward by the specified distance.

Aliases: forward | fd
```

OR

```
Argument:
distance -- a number (integer or float)

Move the turtle forward by the specified distance, in the direction
the turtle is headed.

Example (for a Turtle instance named turtle):

>>> turtle.position()
(0.00, 0.00)
```

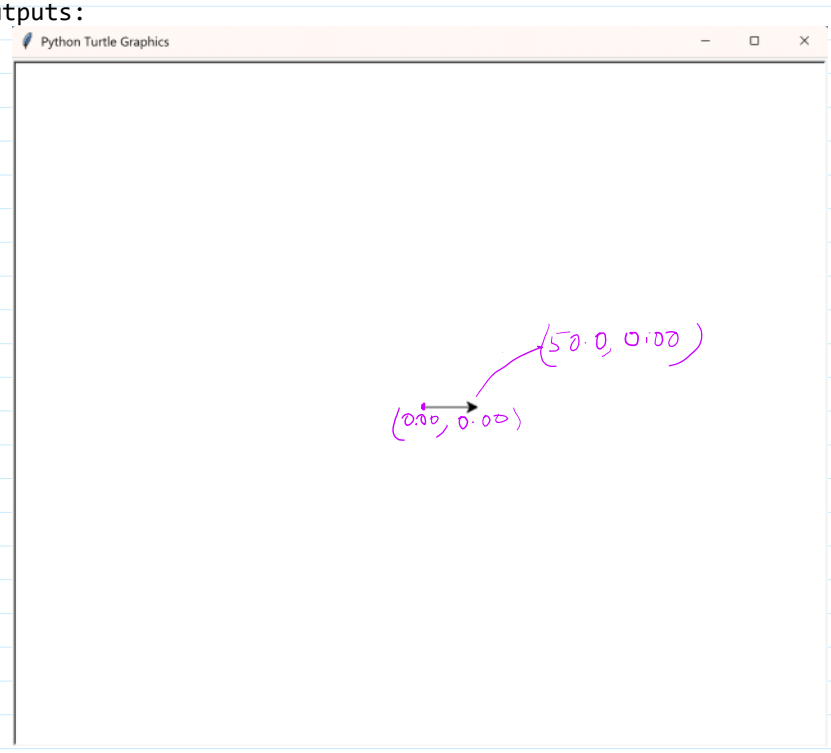
```
>>> turtle.forward(25)
>>> turtle.position()
(25.00,0.00)
>>> turtle.forward(-75)
>>> turtle.position()
(-50.00,0.00)
```

```
() -> Vec2D
Return the turtle's current location (x,y), as a Vec2D-vector.

Aliases: pos | position

No arguments.

Example (for a Turtle instance named turtle):

>>> turtle.pos()
(0.00, 240.00)
```

Example:

```python
import turtle as T
Joe = T.Turtle()
p = Joe.pos()
print(type(p))
print('Position of Joe:', p)
Joe.fd(50)
p = Joe.pos()
print('Position of Joe after moving 50 steps in x axis:', p)
T.done()
```

Outputs:




```
<class 'turtle.Vec2D'>
Position of Joe: (0.00,0.00)
Position of Joe after moving 50 steps in x axis: (50.00,0.00)
```

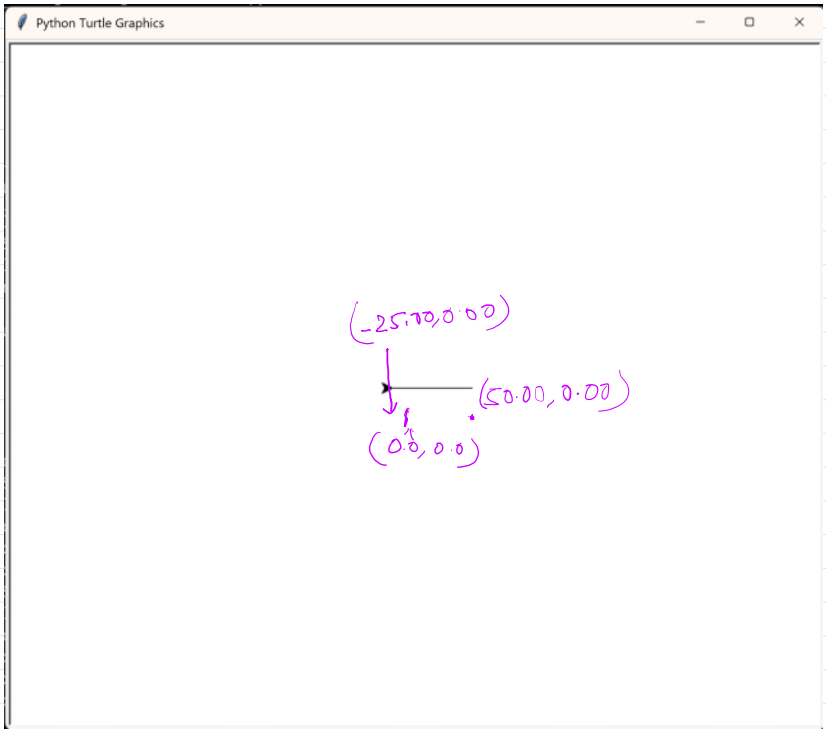
-backward(x): Moves backward x units.

```
def backward(
    self: Self@TNavigator,
    distance: float
) -> None

Move the turtle backward by distance.
Aliases: back | backward | bk
Argument:
distance -- a number
Move the turtle backward by distance, opposite to the direction the
turtle is headed. Do not change the turtle's heading.
Example (for a Turtle instance named turtle):

>>> turtle.position()
(0.00, 0.00)

>>> turtle.backward(30)

>>> turtle.position()
(-30.00, 0.00)
```

Example:

```
import turtle as T
Joe = T.Turtle()
p = Joe.pos()
print(type(p))
print('Position of Joe:', p)
Joe.fd(50)
p = Joe.pos()
print('Position of Joe after moving 50 steps in x axis:', p)
Joe.bk(75)
p = Joe.pos()
print('Position of Joe after moving 75 steps in x axis backward:', p)

T.done()
```

Outputs:




```
Position of Joe: (0.00,0.00)
Position of Joe after moving 50 steps in x axis: (50.00,0.00)
Position of Joe after moving 75 steps in x axis backward: (-25.00,0.00)
```

-right(x): Tones total right by X degrees.

```
def rt(
    self: Self@TNavigator,
    angle: float
) -> None

Turn turtle right by angle units.
Aliases: right | rt
Argument:
angle -- a number (integer or float)
Turn turtle right by angle units. (Units are by default degrees,
but can be set via the degrees() and radians() functions.)
Angle orientation depends on mode. (See this.)
Example (for a Turtle instance named turtle):

>>> turtle.heading()

22.0

>>> turtle.right(45)

>>> turtle.heading()
337.0
```

```
def heading() -> float
```

Return the turtle's current heading.

No arguments.

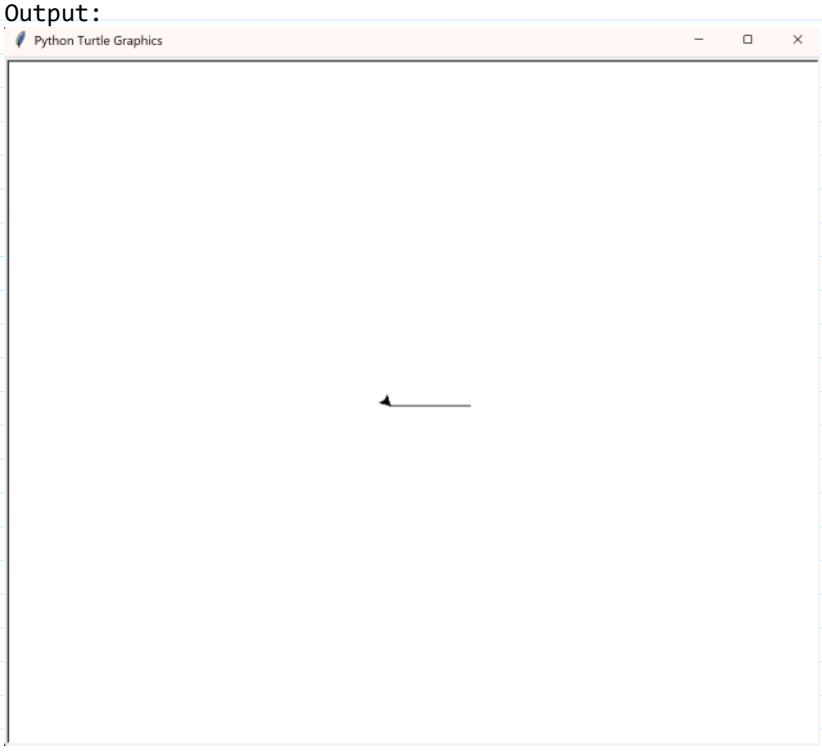
Example (for a Turtle instance named turtle):

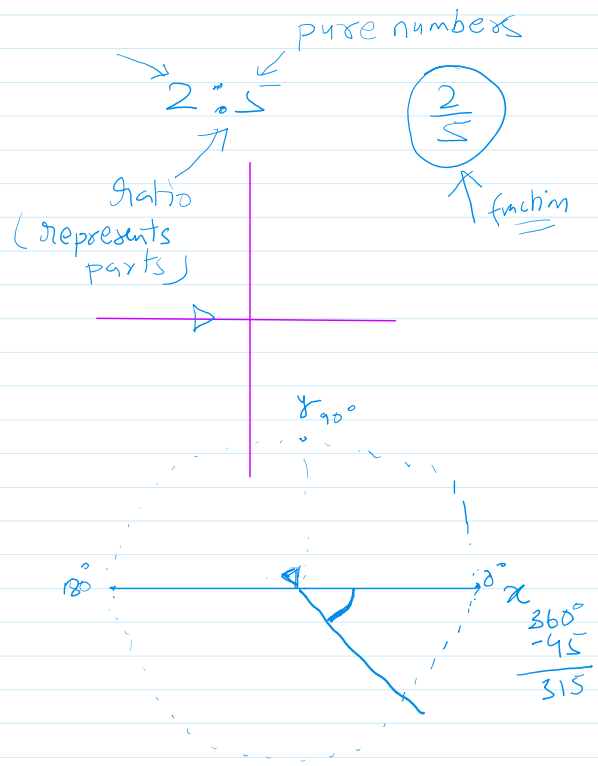
```
>>> turtle.left(67)
>>> turtle.heading()
67.0
```

Example:

```python
import turtle as T
Joe = T.Turtle()
p = Joe.pos()
print(type(p))
print('Position of Joe:', p)
Joe.fd(50)
p = Joe.pos()
print('Position of Joe after moving 50 steps in x axis:', p)
Joe.bk(75)
p = Joe.pos()
print('Position of Joe after moving 75 steps in x axis backward:', p)
print('Current heading:',Joe.heading())
Joe.rt(45)
print('After turning 45 degree right:', Joe.heading())
T.done()
```

Output:






```
<class 'turtle.Vec2D'>
Position of Joe: (0.00,0.00)
Position of Joe after moving 50 steps in x axis: (50.00,0.00)
Position of Joe after moving 75 steps in x axis backward: (-25.00,0.00)
Current heading: 0.0
After turning 45 degree right: 315.0
```

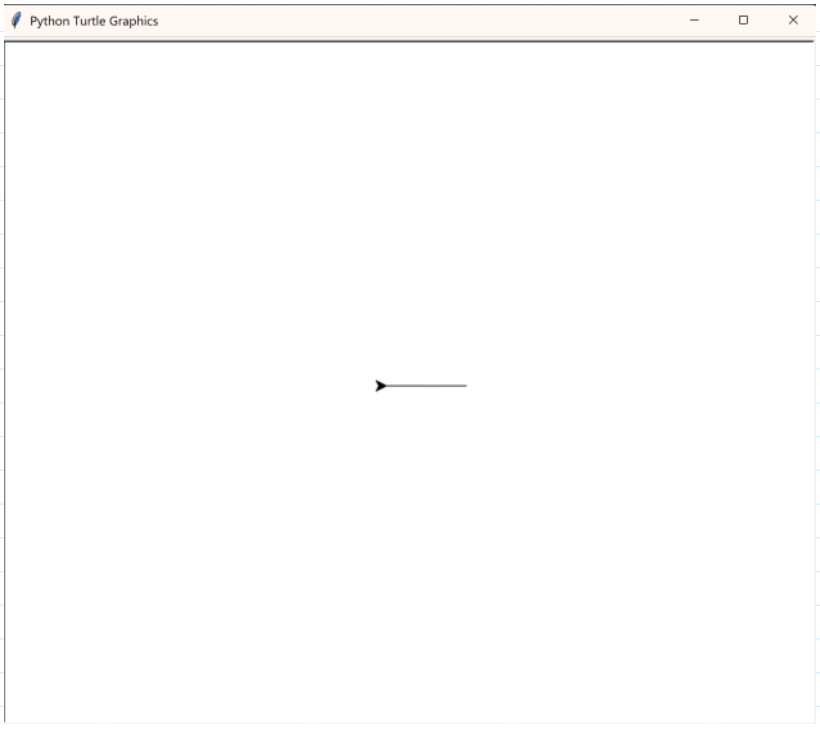
-left(x)=> Tons turtle left by X degrees.

```
def left(angle: float) -> None

Turn turtle left by angle units.
Aliases: left | lt
Argument:
angle -- a number (integer or float)
Turn turtle left by angle units. (Units are by default degrees,
but can be set via the degrees() and radians() functions.)
Angle orientation depends on mode. (See this.)
Example (for a Turtle instance named turtle):

>>> turtle.heading()
22.0

>>> turtle.left(45)

>>> turtle.heading()
67.0
```

Example:

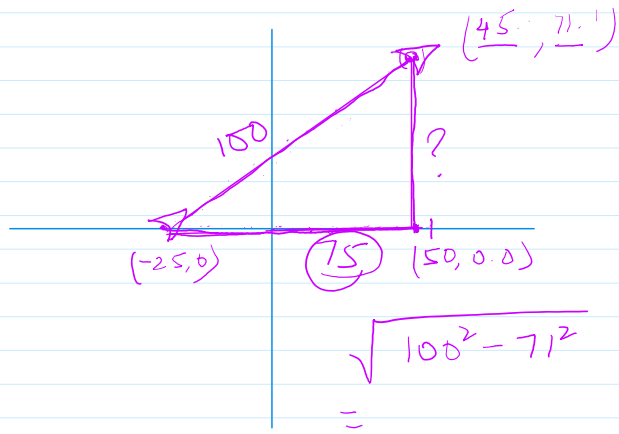
```
import turtle as T
Joe = T.Turtle()
p = Joe.pos()
print(type(p))
print('Position of Joe:', p)
Joe.fd(50)
p = Joe.pos()
print('Position of Joe after moving 50 steps in x axis:', p)
Joe.bk(75)
p = Joe.pos()
print('Position of Joe after moving 75 steps in x axis backward:', p)
print('Current heading:',Joe.heading())
Joe.rt(45)
print('After turning 45 degree right:', Joe.heading())
Joe.left(45)
print('After turning 45 degree left:', Joe.heading())
T.done()
```

Outputs:




```
<class 'turtle.Vec2D'>
Position of Joe: (0.00,0.00)
Position of Joe after moving 50 steps in x axis: (50.00,0.00)
Position of Joe after moving 75 steps in x axis backward: (-25.00,0.00)
Current heading: 0.0
After turning 45 degree right: 315.0
After turning 45 degree left: 0.0
```

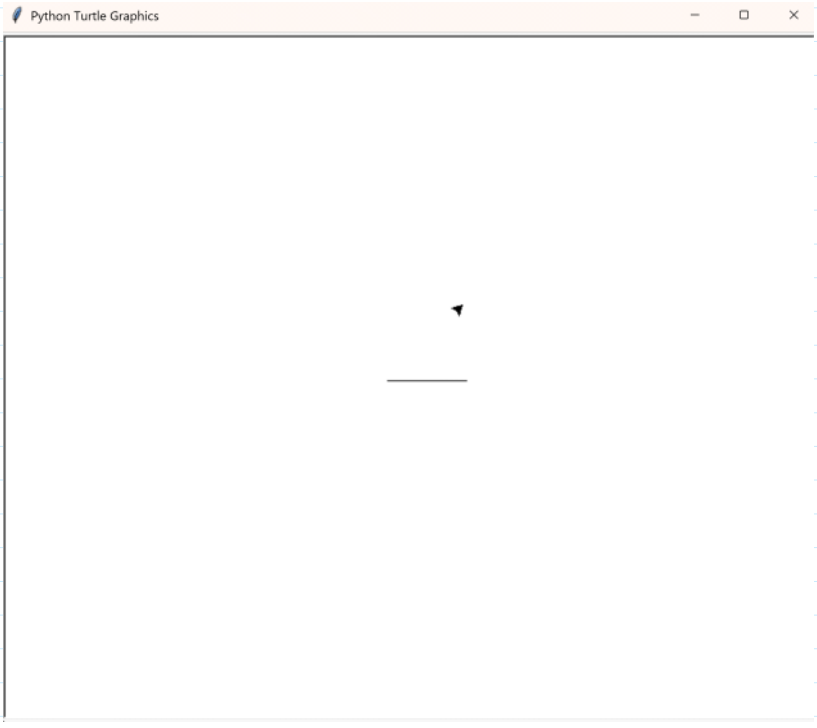
-penup(): Stops Drawing
-pendown(): Starts Drawing




```python
import turtle as T
Joe = T.Turtle()
p = Joe.pos()
print(type(p))
print('Position of Joe:', p)
Joe.fd(50)
p = Joe.pos()
print('Position of Joe after moving 50 steps in x axis:', p)
Joe.bk(75)
p = Joe.pos()
print('Position of Joe after moving 75 steps in x axis backward:', p)
print('Current heading:',Joe.heading())
Joe.rt(45)
```

```
print('After turning 45 degree right:', Joe.heading())
Joe.left(45)
print('After turning 45 degree left:', Joe.heading())
Joe.penup()
Joe.left(45)
print('After turning 45 degree left:', Joe.heading())
Joe.fd(100)
print('Position of Joe after moving 100 steps towards 45 degree:', p)
print('Current position of Joe is:', Joe.pos())
T.done()
```

Output:



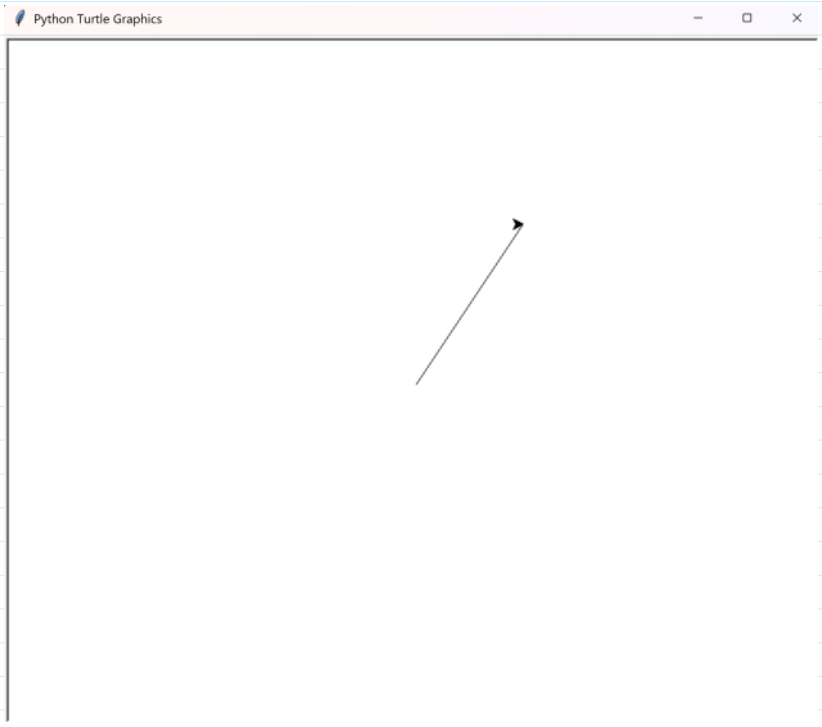

```
<class 'turtle.Vec2D'>
Position of Joe: (0.00,0.00)
Position of Joe after moving 50 steps in x axis: (50.00,0.00)
Position of Joe after moving 75 steps in x axis backward: (-25.00,0.00)
Current heading: 0.0
After turning 45 degree right: 315.0
After turning 45 degree left: 0.0
After turning 45 degree left: 45.0
Position of Joe after moving 100 steps towards 45 degree: (-25.00,0.00)
Current position of Joe is: (45.71,70.71)
>>> import math as M
>>> M.sqrt(100**2 - 71**2)
70.42016756583301
```

↑ Approx value taken to calculate the y-axis value.

goto(x, y): Moves to coordinate(x, y)

```
import turtle as T
jack = T.Turtle()
print('current position of jack:',jack.pos())
jack.goto(100, 150)
print('current position of jack:',jack.pos())
T.done() #to complete the execution of window
```

Output:

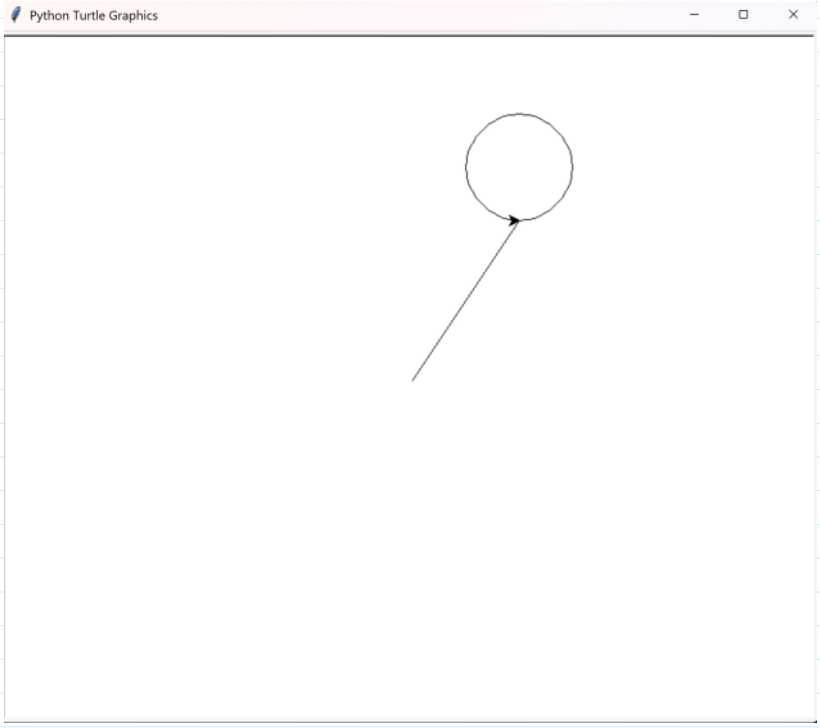



```
current position of jack: (0.00,0.00)
current position of jack: (100.00,150.00)
```

circle(r): Draws a Circle

```python
import turtle as T
jack = T.Turtle()
print('current position of jack:',jack.pos())
jack.goto(100, 150)
print('current position of jack:',jack.pos())
jack.circle(50) #radius = 10 units
print('current position of jack:',jack.pos())
T.done() #to complete the execution of window
```

Output:



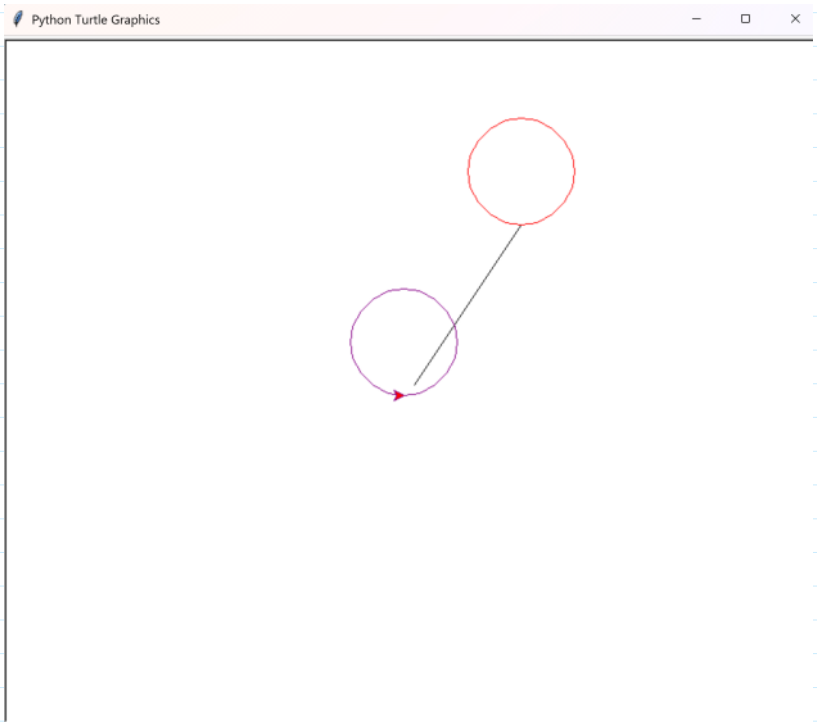

```
current position of jack: (0.00,0.00)
current position of jack: (100.00,150.00)
current position of jack: (100.00,150.00)
```

After drawing circle Jack has retured to the same place.

color('red')

```
import turtle as T
jack = T.Turtle()
print('current position of jack:',jack.pos())
jack.goto(100, 150)
print('current position of jack:',jack.pos())
jack.color('red')
jack.circle(50) #radius = 10 units
print('current position of jack:',jack.pos())
jack.pencolor('purple')
jack.penup()
jack.goto(-10, -10)
jack.pendown()
jack.circle(50)
print('current position of jack:',jack.pos())
T.done() #to complete the execution of window
```

Output:



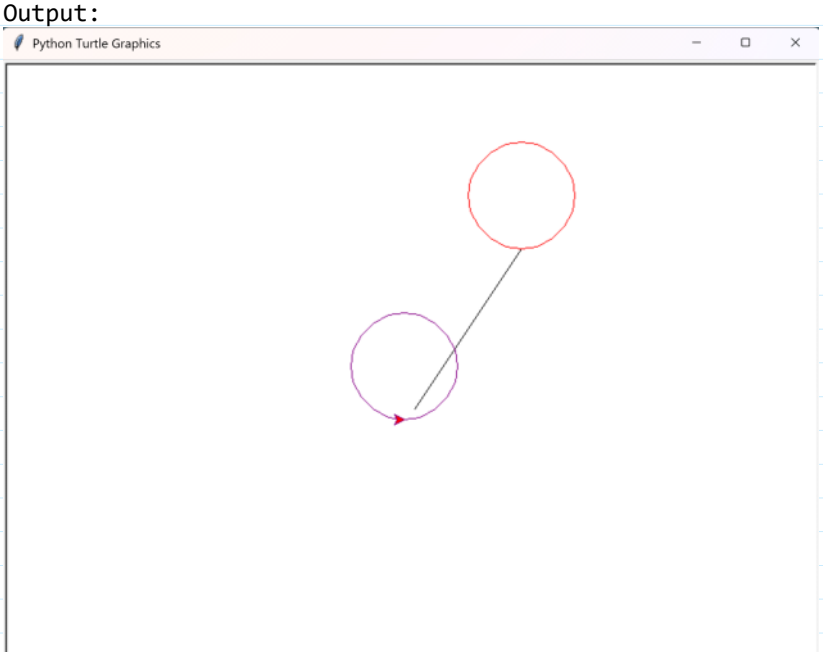

```
current position of jack: (0.00,0.00)
current position of jack: (100.00,150.00)
current position of jack: (100.00,150.00)
current position of jack: (-10.00,-10.00)
```

speed(1-10): sets the turtle speed
Syntax of turtle.speed():

```
def speed(speed: None = None) -> int: ...
def speed(speed: _Speed) -> None: ...
Return or set the turtle's speed.

Optional argument:
speed -- an integer in the range 0..10 or a speedstring (see below)

Set the turtle's speed to an integer value in the range 0 .. 10.
If no argument is given: return current speed.

If input is a number greater than 10 or smaller than 0.5,
speed is set to 0.
Speedstrings are mapped to speedvalues in the following way:
    'fastest' : 0
    'fast' : 10
    'normal' : 6
    'slow' : 3
    'slowest' : 1
speeds from 1 to 10 enforce increasingly faster animation of
line drawing and turtle turning.

Attention:
speed = 0 : no animation takes place. forward/back makes turtle jump
and likewise left/right make the turtle turn instantly.

Example (for a Turtle instance named turtle):

>>> turtle.speed(3)
```

```python
import turtle as T
jack = T.Turtle()
print('current position of jack:',jack.pos())
currentSpeed = jack.speed()
print('Default speed of the turtle Jack:', currentSpeed)
jack.speed(1) #slowest
print('Now the speed of Jack the turtle:', jack.speed())
jack.goto(100, 150)
print('current position of jack:',jack.pos())
jack.color('red')
jack.circle(50) #radius = 10 units
print('current position of jack:',jack.pos())
jack.pencolor('purple')
jack.penup()
jack.goto(-10, -10)
jack.pendown()
jack.circle(50)
print('current position of jack:',jack.pos())
T.done() #to complete the execution of window
```

Output:




```
current position of jack: (0.00,0.00)
Default speed of the turtle Jack: 3
Now the speed of Jack the turtle: 1
current position of jack: (100.00,150.00)
current position of jack: (100.00,150.00)
current position of jack: (-10.00,-10.00)
```

default speed

slowest speed.

```
 shape('turtle'): Turtle icon shape
def shape(name: None = None) -> str: ...
def shape(name: str) -> None: ...
Set turtle shape to shape with given name / return current shapename.

Optional argument:
name -- a string, which is a valid shapename

Set turtle shape to shape with given name or, if name is not given,
return name of current shape.
Shape with name must exist in the TurtleScreen's shape dictionary.
Initially there are the following polygon shapes:
'arrow', 'turtle', 'circle', 'square', 'triangle', 'classic'.
```

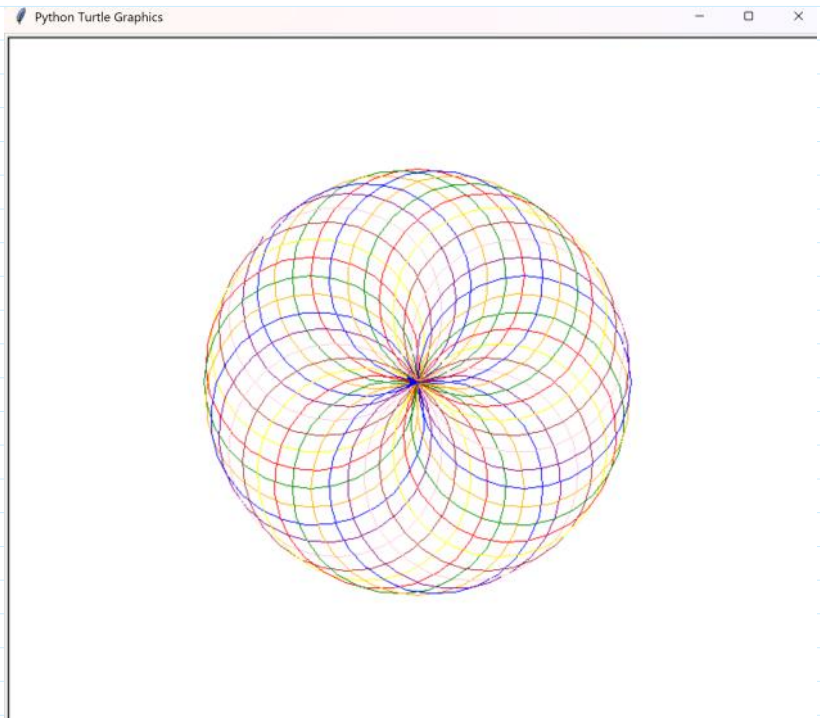
To learn about how to deal with shapes see Screen-method register_shape.

Example (for a Turtle instance named turtle):

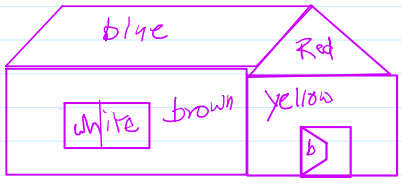
```
>>> turtle.shape()
'arrow'
>>> turtle.shape("turtle")
>>> turtle.shape()
'turtle'
```

```python
import turtle as T
kia = T.Turtle()
colors = ['red', 'green', 'orange', 'blue', 'purple', 'pink', 'brown', 'yellow']
for i in range(36):
    kia.color(colors[i % len(colors)])
    kia.circle(100)
    kia.left(10)

T.done()
```

Output:



WAP to help your turtle to draw a house, like:




```
fdmove = [40, 50, . . . . ]
turn = [90, 60, 120, 30]
for index in range(len(turn)):
    jini.fd(fdmove[index])
```

teleport():
turtle.teleport(*x, y=None, *, fill_gap=False*)

Parameters:

- **x – a number or None**
- **y – a number or None**

- fill_gap – a boolean

Move turtle to an absolute position. Unlike goto(x, y), a line will not be drawn. The turtle's orientation does not change. If currently filling, the polygon(s) teleported from will be filled after leaving, and filling will begin again after teleporting. This can be disabled with fill_gap=True, which makes the imaginary line traveled during teleporting act as a fill barrier like in goto(x, y).

```
import turtle as t
harry = t.Turtle()
pos = harry.pos()
print(pos)
harry.color('blue', 'red')
harry.begin_fill()
harry.teleport(60)
pos = harry.pos()
print(pos)
harry.teleport(y= 50)
pos = harry.pos()
print(pos)
harry.teleport(20, 30)
pos = harry.pos()
print(pos)
harry.teleport(40, 60)
pos = harry.pos()
print(pos)
harry.teleport(-40, 60)
pos = harry.pos()
# harry.home()
# print(harry.pos())
harry.end_fill()

t.done()
```

turtle.**setheading**(*to_angle*)
turtle.**seth**(*to_angle*)

Parameters: **to_angle** – a number (integer or float)

Set the orientation of the turtle to *to_angle*. Here are some common directions in degrees:

standard mode	logo mode
0 - east	0 - north
90 - north	90 - east
180 - west	180 - south
270 - south	270 - west

```
>>> turtle.setheading(90)
>>> turtle.heading()
90.0
```

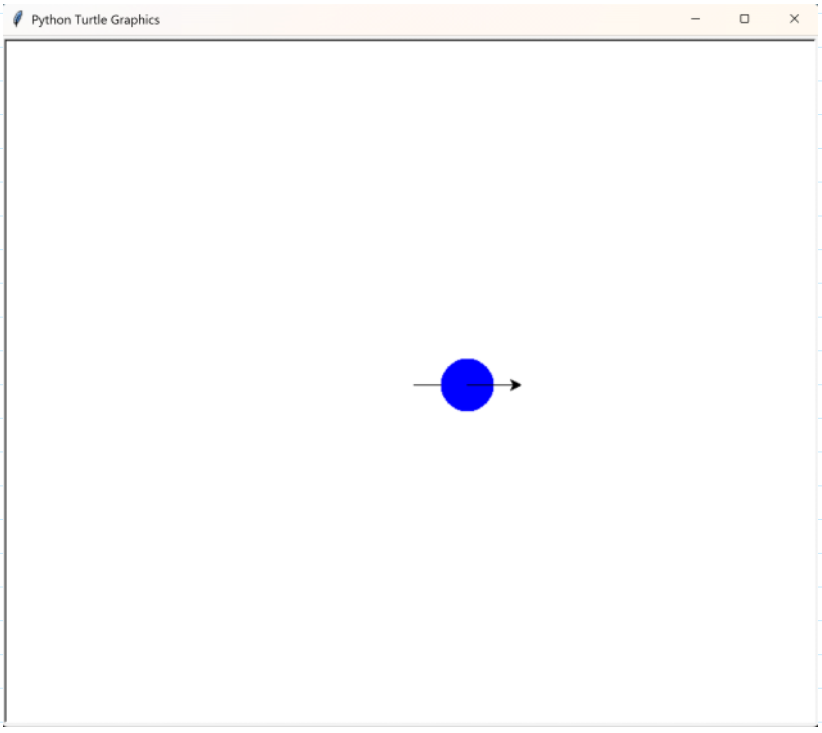
```
import turtle as t
import os
os.system('cls') # to clear terminal window
harry = t.Turtle()
p = harry.heading()
print(p)
harry.seth(270)
t.done()
```

`turtle.dot(size=None, *color)`

Parameters:
- **size** – an integer >= 1 (if given)
- **color** – a colorstring or a numeric color tuple

Draw a circular dot with diameter *size*, using *color*. If *size* is not given, the maximum of pensize+4 and 2*pensize is used.

```
import turtle as t
import os
os.system('cls') # to clear terminal window
harry = t.Turtle()
harry.fd(50); harry.dot(50,'blue'); harry.fd(50)
print(harry.pos())
t.done()
```



stamp()

`turtle.stamp()`

Stamp a copy of the turtle shape onto the canvas at the current turtle position. Return a stamp_id for that stamp, which can be used to delete it by calling `clearstamp(stamp_id)`.

```
>>> turtle.color("blue")
>>> stamp_id = turtle.stamp()
>>> turtle.fd(50)
```

Copy

`turtle.clearstamp(stampid)`

Parameters: **stampid** – an integer, must be return value of previous `stamp()` call

`turtle.clearstamps(n=None)`

Parameters: **n** – an integer (or `None`)

Delete all or first/last *n* of turtle's stamps. If *n* is `None`, delete all stamps, if *n* > 0 delete first *n* stamps, else if *n* < 0 delete last *n* stamps.

```
import turtle as t
import os
os.system('cls') # to clear terminal window
harry = t.Turtle()
```

```
harry.speed(1)
harry.color('blue')
stamp_id = harry.stamp()
print(stamp_id)
harry.fd(100)
stamp_id2 = harry.stamp()
harry.seth(90)
harry.fd(50)
print(stamp_id2)
print(harry.pos())
harry.clearstamp(stamp_id)
harry.clearstamps()
t.done()
```

turtle.undo()

Undo (repeatedly) the last turtle action(s). Number of available undo actions is determined by the size of the undobuffer.

```
>>> for i in range(4):
...     turtle.fd(50); turtle.lt(80)
...
>>> for i in range(8):
...     turtle.undo()
```

```
import turtle as t
import os
os.system('cls') # to clear terminal window
harry = t.Turtle()
harry.speed(1)
for i in range(5):
    harry.fd(50)
    harry.lt(72)

for i in range(5):
    harry.undo()

t.done()
```

Methods of TurtleScreen/Screen

81 → $1 \times 16^0 = 1$, $8 \times 16^1 = 128$, 129

(method) def bgcolor(color: _Color) -> None

() -> _AnyColor

Set or return backgroundcolor of the TurtleScreen.

Arguments (if given): a color string or three numbers in the range 0..colormode or a 3-tuple of such numbers.

Example (for a TurtleScreen instance named screen):

4F → $15 \times 16^0 = 15$, $4 \times 16^1 = 64$, 79

```
>>> screen.bgcolor("orange")

>>> screen.bgcolor()
'orange'

>>> screen.bgcolor(0.5,0,0.5)
```

C2 → $2 \times 16^0 = 2$, $12 \times 16^1 = 192$, 194

194/255=0.7608

0.7607843137254902

```
>>> screen.bgcolor()
'#800080'
```

79/255 = 0.3098 0.30980392156862746

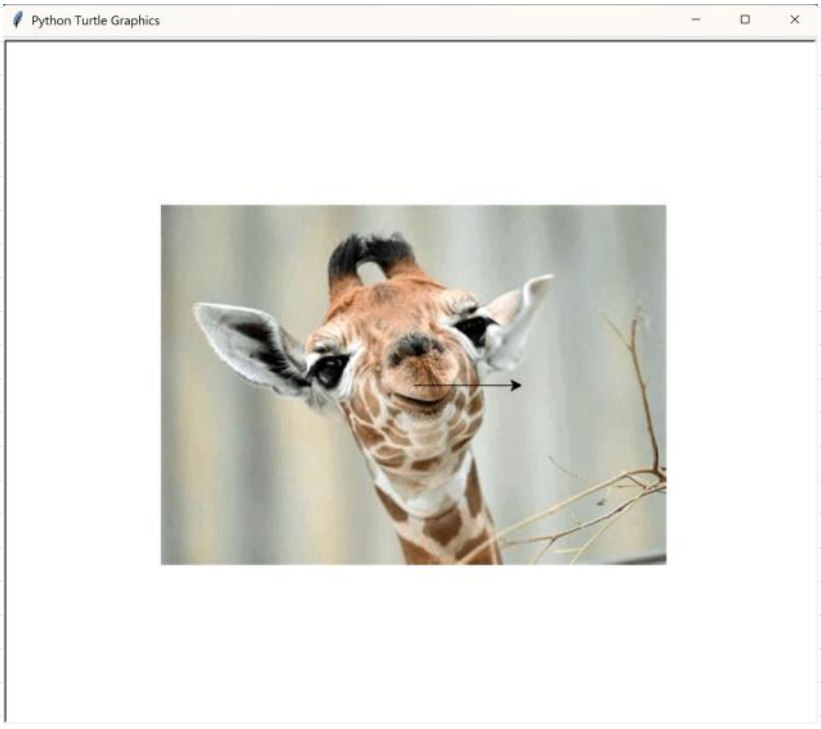
129/255 = 0.5059 0.5058823529411764

```python
import turtle as T
# Most of the examples in this section refer to a TurtleScreen
instance called screen.
canvas = T.getcanvas()
screen = T.TurtleScreen(canvas)
screen.bgcolor("#C24F81")
print(screen.bgcolor())
hari = T.RawTurtle(screen)
hari.fd(100)
screen.mainloop()
T.done()
```

```python
def bgpic(picname: None = None) -> str: ...
def bgpic(picname: str) -> None: ...

Set background image or return name of current backgroundimage.
Optional argument:
picname -- a string, name of a gif-file or "nopic".
If picname is a filename, set the corresponding image as
background.
If picname is "nopic", delete backgroundimage, if present.
If picname is None, return the filename of the current
backgroundimage.
Example (for a TurtleScreen instance named screen):

>>> screen.bgpic()
'nopic'

>>> screen.bgpic("landscape.gif")

>>> screen.bgpic()
'landscape.gif'
```

```python
import turtle as T
# Most of the examples in this section refer to a TurtleScreen
instance called screen.
canvas = T.getcanvas()
screen = T.TurtleScreen(canvas)
# screen.bgcolor("#C24F81")
# print(screen.bgcolor())
screen.bgpic('TurtleProgramming/smilinggiraffe.gif')
hari = T.RawTurtle(screen)
hari.fd(100)
screen.mainloop()
T.done()
```

Output:



Using screen events on turtle
Screen events in pythons turtle module allow responding to user input. (keyboard and mouse action.) By binding functions (event handlers) to these events. The core methods for screen events include onkey(), onkeypress(), onkeyrelease() for keyboard and onscreenclick(), onclick(), onrelease() for mouse. The listen() method is required to ensure the detects events.

Keyboard Event Example:
Use onkey() to bind a function to a key press event. For example, moving the turtle with arrow keys.

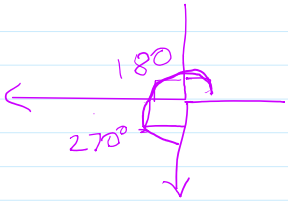
```
def listen(
    xdummy: float | None = None,
    ydummy: float | None = None
) -> None
```

Set focus on TurtleScreen (in order to collect key-events)

No arguments.
Dummy arguments are provided in order to be able to pass listen to the onclick method.

Example (for a TurtleScreen instance named screen):

```
>>> screen.listen()
```




```python
import turtle as T
def move_left():
    T.setheading(180)
    T.fd(50)
def move_right():
    T.setheading(0)
    T.fd(50)
```

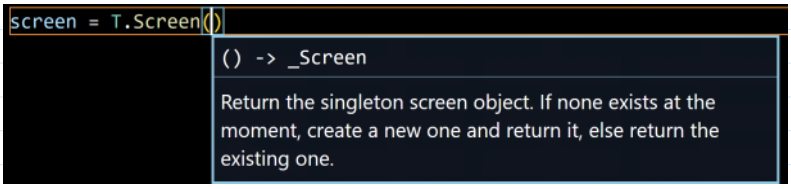
```
def move_up():
    T.setheading(90)
    T.fd(50)
def move_down():
    T.setheading(270)
    T.fd(50)
screen = T.Screen()
screen.onkey(move_left, "Left")
screen.onkey(move_right, "Right")
screen.onkey(move_up, "Up")
screen.onkey(move_down, "Down")
screen.listen()
T.mainloop() #or T.done()
```

This binds the arrow keys to their respective functions;

listen() is called so screen events are captured.

Output:
Write the code in your (vscode, pycharm, anaconda…)ide and execute and have fun.

Mouse Event Example
For handling mouse clicks anywhere on the screen:




```
import turtle as T
def goto_click(x, y):
    T.goto(x, y)

screen = T.Screen()
screen.onscreenclick(goto_click)
T.done()
```

Output:
Write the code in your (vscode, pycharm, anaconda…)ide and execute and have fun.

Draw a shape on Keypress

```
import turtle as T
def draw_square():
    for _ in range(4):
        T.fd(100)
        T.left(90)

screen = T.Screen()
screen.onkeypress(draw_square, "space")
screen.listen()
T.done()
```

Output:
Write the code in your (vscode, pycharm, anaconda…)ide and execute and have fun.

onkey() or onkeypress()

```
def onkey(
    fun: () -> object,
    key: str
) -> None
```

Bind fun to key-release event of key.

Home works:
WAP using turtle module, press 'c' to draw a circle, press 's' to draw a square, press 't' for triangle.